

Compiler Design

Complete MCQ-Oriented Notes

IBPS SO (IT Officer) Examination - Complete Preparation Series

100% Syllabus Coverage - MCQ-Oriented - Zero Filler

Compiled by Asabhya Maanav

Table of Contents

Compiler Design - Orientation for CIL MT Systems CBT	5
How to use this book	5
1. Compiler Overview	6
1.1 Compiler vs Interpreter vs Assembler	6
1.2 Phases of a Compiler	6
1.3 Analysis (Front End) vs Synthesis (Back End)	7
1.4 Symbol Table and Error Handler	8
1.5 Passes: Single-Pass vs Multi-Pass	8
1.6 Bootstrapping and Cross-Compiler (Awareness)	8
2. Lexical Analysis	9
2.1 Role of the Lexical Analyzer (Scanner)	9
2.2 Tokens, Patterns, Lexemes	9
2.3 Regular Expressions for Token Specification	9
2.4 Finite Automata for Recognizing Tokens	9
2.5 Input Buffering	10
2.6 LEX / Flex Tool	10
2.7 Symbol Table Interaction	10
3. Grammars for Parsing	11
3.1 Context-Free Grammars (CFG)	11
3.2 Parse Trees and Derivations	11
3.3 Ambiguity, Left Recursion, Left Factoring	11
4. Top-Down Parsing	13
4.1 Recursive Descent Parsing	13
4.2 Predictive Parsing LL(1)	13
4.3 Backtracking Parsers	14
5. Bottom-Up Parsing	15
5.1 Handles and Handle Pruning	15
5.2 Shift-Reduce Parsing and Conflicts	15
5.3 LR Parsing Family	15
5.4 Comparison of LL, SLR, LALR, CLR Power	16
5.5 Operator Precedence Parsing	16
5.6 YACC / Bison Tool	16

6. Syntax-Directed Translation (SDT)	17
6.1 Syntax-Directed Definitions (SDD)	17
6.2 Synthesized vs Inherited Attributes	17
6.3 Annotated Parse Trees	17
6.4 S-Attributed and L-Attributed Definitions	17
6.5 Translation Schemes	17
6.6 Dependency Graphs	18
7. Semantic Analysis	19
7.1 Type Checking and Type Systems	19
7.2 Type Conversions / Coercion	19
7.3 Scope Checking	19
7.4 Symbol Table Organization and Operations	19
8. Runtime Environments	20
8.1 Storage Organization	20
8.2 Activation Records (Stack Frames)	20
8.3 Stack Allocation; Heap Management	20
8.4 Access to Non-Local Names	20
8.5 Parameter Passing	21
9. Intermediate Code Generation	22
9.1 Intermediate Representations	22
9.2 Three-Address Code (TAC)	22
9.3 Representations of TAC: Quadruples, Triples, Indirect Triples	22
9.4 Translation of Expressions and Assignments	22
9.5 Translation of Control Flow and Boolean Expressions	23
9.6 Backpatching	23
9.7 Translation of Arrays and Function Calls	23
10. Local & Machine-Independent Optimization	24
10.1 Basic Blocks and Flow Graphs	24
10.2 Local Optimizations	24
10.3 Loop Optimizations	25
10.4 Peephole Optimization	25
11. Data Flow Analysis	26
11.1 Reaching Definitions	26
11.2 Live Variable (Liveness) Analysis	26
11.3 Available Expressions	26

11.4 Constant Propagation Analysis	26
11.5 Use of CSE via Data Flow	26
12. Code Generation	27
12.1 Issues in Code Generation	27
12.2 Register Allocation and Spilling	27
12.3 Simple Code Generation from TAC	27
12.4 DAG-Based Local Code Generation	27
13. Exam Focus / Practice Checklist	28
13.1 Phase Identification	28
13.2 FIRST and FOLLOW Computation	28
13.3 LL(1) / LR Conflict Detection	28
13.4 Parser-Type Power Comparison	28
13.5 Synthesized vs Inherited	28
13.6 Quadruples/Triples	28
13.7 Number of Tokens	28
13.8 Definitions and One-Liners	28

Compiler Design - Orientation for CIL MT Systems CBT

- **Compiler Design** is the study of how a **source program** written in a high-level language is translated, phase by phase, into an equivalent **target program** (machine/assembly code) while detecting errors.
- For the CIL MT Systems CBT the questions are **objective (MCQ)** and range from **conceptual to moderate** difficulty, lighter than GATE but built on the SAME core facts.
- The **6 phases** of a compiler in order are: **Lexical Analysis**, **Syntax Analysis**, **Semantic Analysis**, **Intermediate Code Generation**, **Code Optimization**, and **Code Generation**.
- High-frequency MCQ zones: **phase identification**, **FIRST/FOLLOW**, **parser power comparison**, **synthesized vs inherited attributes**, **quadruples/triples**, and **token counting**.
- Memory hook for the phases: **"Lovely Students Shouldlone Optimize Code"** = Lexical, Syntax, Semantic, Intermediate, Optimize, Code-gen.

How to use this book

- Each "###" section follows the official syllabus order so nothing is skipped at the atomic level.
- Every keyword, number, formula, and core answer is printed in **red** so the eye can drill straight to the examinable fact.
- Each topic ends with a **CBT Trap** callout naming the exact confusions examiners exploit.

1. Compiler Overview

1.1 Compiler vs Interpreter vs Assembler

- A **compiler** translates the ENTIRE source program into target code in one shot BEFORE execution; errors of the whole program are reported together.
- An **interpreter** translates and executes the program **statement by statement** (line by line); it stops at the first runtime error.
- An **assembler** translates **assembly language** (mnemonics) into **machine code** (object code); it is a special translator with a near 1:1 mapping.
- A **loader** loads the executable into memory; a **linker** combines multiple object modules and resolves external references.
- A **preprocessor** handles macros and file inclusion (e.g., #include, #define) BEFORE the compiler proper runs.

Comparison Table

Property	Compiler	Interpreter	Assembler
Input	HLL source	HLL source	Assembly
Translation unit	Whole program	One statement	One instruction
Speed of execution	Fast	Slow	Fast
Error reporting	All at once	First error then halt	Per line
Memory	Needs space for object code	Lower	Low
Example	C, C++	Python, JS (classic)	NASM, MASM

Memory Trick

- **Compiler = whole**, **Interpreter = line**. "Interpreter Iterates Individually."
- Compiled programs run faster because translation overhead is paid **once**; interpreted programs re-translate every run.

CBT Trap

- Examiners ask "which is faster to **debug**?" -> **interpreter** (immediate feedback); "which produces faster **execution**?" -> **compiler**.
- **Java** is BOTH: source -> **bytecode** (compiler) -> executed by **JVM** (interpreter/JIT). Do not call it purely compiled or interpreted.
- An assembler is NOT a compiler; mapping is essentially **one-to-one**, not one-to-many.

1.2 Phases of a Compiler

- The compiler is divided into **6** phases; the **symbol table** and **error handler** interact with ALL of them and are NOT counted as phases.
- The phases are commonly grouped into **Analysis (front end)** and **Synthesis (back end)**.

1.2.1 Lexical Analysis

- Reads the **character stream** and groups characters into **tokens**; removes whitespace and comments.
- Output: a stream of **tokens** of the form <token-name, attribute-value>.
- Detects errors like **illegal characters** / invalid tokens (e.g., @ in C identifier).

1.2.2 Syntax Analysis

- Also called **parsing**; groups tokens into grammatical phrases per a **Context-Free Grammar (CFG)**.
- Output: a **parse tree** / **syntax tree**.
- Detects **syntactic errors** (e.g., missing `;`, unbalanced parentheses).

1.2.3 Semantic Analysis

- Checks **meaning**: **type checking**, declaration-before-use, scope, operand compatibility.
- Output: an **annotated/decorated syntax tree**.
- Detects **semantic errors** (e.g., **type mismatch**, undeclared variable, array index type error).

1.2.4 Intermediate Code Generation

- Produces a machine-independent **intermediate representation (IR)**, most often **three-address code (TAC)**.
- Easy to **optimize** and to **retarget** to many machines.

1.2.5 Code Optimization

- Improves IR to run **faster** and/or use **less space** WITHOUT changing meaning.
- **Machine-independent** optimization happens here; **machine-dependent** happens after code generation.

1.2.6 Code Generation

- Translates IR into **target machine code**; handles **instruction selection**, **register allocation**, and **instruction ordering**.
- Output: **relocatable machine code** or assembly.

Phase -> Error Table (very high yield)

Phase	Detects	Example Error
Lexical	Invalid token	1abc, @, unterminated string
Syntax	Grammar violation	missing <code>;</code> , <code>int x = ;</code>
Semantic	Type/scope	adding <code>int + string</code> , undeclared <code>y</code>
Intermediate	(none new)	-
Optimization	(none new)	-
Code gen	machine limits	register overflow handled

Memory Trick

- Errors caught order: **Lexical -> Syntax -> Semantic**. "Letters, then Sentences, then Meaning."

CBT Trap

- **Missing semicolon** is a **syntax** error, NOT lexical.
- **Type mismatch** and **undeclared variable** are **semantic** errors, NOT syntax.
- The symbol table is created in **lexical** analysis and used through **code generation**; it is not a phase.

1.3 Analysis (Front End) vs Synthesis (Back End)

- **Front end (Analysis)** = **Lexical + Syntax + Semantic + Intermediate code generation**; it depends on the **source language**, not the target machine.
- **Back end (Synthesis)** = **Code optimization (machine-dependent part) + Code generation**; it depends on the **target machine**.

- This split enables **retargeting**: one front end + many back ends, or many front ends + one back end (**m+n** effort instead of **m*n**).

CBT Trap

- **Intermediate code generation** belongs to the **front end** even though it sounds "middle"; the front end ends at IR.
- Machine-INDEPENDENT optimization is sometimes called the **middle end**.

1.4 Symbol Table and Error Handler

- The **symbol table** is a data structure storing **identifiers** and their attributes: **name, type, scope, memory location, size**.
- It is accessed/updated by **all phases**; common implementations: **hash table** (avg **O(1)** lookup), **binary search tree, linked list**.
- The **error handler** reports errors and performs **error recovery** so compilation can continue.
- Recovery strategies: **panic mode, phrase-level, error productions, global correction**.

CBT Trap

- **Panic mode** discards input symbols until a **synchronizing token** (like **;**) is found - simplest, cannot enter infinite loop.
- **Hash table** symbol-table average lookup is **O(1)**; do not say $O(\log n)$ (that is BST).

1.5 Passes: Single-Pass vs Multi-Pass

- A **pass** is one complete scan of the source (or its IR) from start to end.
- **Single-pass**: all phases done in one scan; **fast**, low memory, but limited optimization (e.g., classic Pascal).
- **Multi-pass**: multiple scans; allows better **optimization** and **forward references**; needs more memory.

CBT Trap

- Number of **phases** (**6**) is fixed and INDEPENDENT of number of **passes**; a single pass can still contain all phases.

1.6 Bootstrapping and Cross-Compiler (Awareness)

- **Bootstrapping**: writing a compiler for a language **in that same language** (a small core compiler builds a bigger one).
- A **cross-compiler** runs on one machine (**host**) but generates code for a **different machine** (**target**).
- **T-diagram** notation describes source language, target language, and implementation language of a translator.

CBT Trap

- Cross-compiler: **host != target**. A self/native compiler has **host == target**.

2. Lexical Analysis

2.1 Role of the Lexical Analyzer (Scanner)

- The **lexical analyzer (scanner)** reads the raw **character stream** and produces a stream of **tokens** for the parser.
- Secondary tasks: strip **whitespace** and **comments**, correlate **error messages** with line numbers, expand macros, and interact with the **symbol table**.
- It is usually a **subroutine** called by the parser ("get next token") rather than a separate pass.

Memory Trick

- Scanner = "**character -> token**"; Parser = "**token -> tree**".

2.2 Tokens, Patterns, Lexemes

- A **token** is a category/class of lexical units, e.g., **identifier**, **keyword**, **number**, **operator**.
- A **pattern** is the **rule** (often a regular expression) describing the set of lexemes a token can match.
- A **lexeme** is the **actual sequence of characters** in the source matching a pattern.
- Example: for `int count = 10;` -> tokens are **keyword (int)**, **id (count)**, **operator (=)**, **number (10)**, **separator (;)**; the lexeme for the id token is `count`.

Token vs Pattern vs Lexeme Table

Term	Meaning	Example	
Token	Class/name	id, num, relop	
Pattern	Rule	letter(letter	digit)*
Lexeme	Actual text	count, 10, >=	

CBT Trap

- Many lexemes can map to **one** token (e.g., `x`, `count`, `total` are all the **id** token).
- Count **tokens** not **lexemes** when asked "number of tokens"; keywords, identifiers, operators, separators each count.

2.3 Regular Expressions for Token Specification

- A **regular expression (regex)** specifies the patterns of tokens; tokens form a **regular language**.
- Operations (precedence high->low): **Kleene star *** > **concatenation** > **union |**.
- Identities: **$r|s = s|r$** (commutative union), **$(r^*)^* = r^*$** , **$re = er = r$** where e is epsilon.
- Examples: `identifier = letter (letter | digit)*`; `integer = digit digit* = digit+`.

CBT Trap

- Regular expressions cannot count** / cannot match **balanced parentheses** -> that needs a **CFG** (this is why lexical < syntax in power).
- $r+ = \text{one or more} = r r^*$** ; **$r^* = \text{zero or more}$** ; **$r? = \text{zero or one}$** .

2.4 Finite Automata for Recognizing Tokens

- Regular expressions are converted to **finite automata** to recognize tokens; the scanner is essentially a **DFA**.

- **NFA** (Nondeterministic) allows **epsilon** moves and multiple transitions; **DFA** (Deterministic) has exactly **one** transition per symbol and **no** epsilon.
- Pipeline: **Regex -> NFA (Thompson's construction) -> DFA (subset construction) -> minimized DFA**.
- A DFA with n NFA states can have up to 2^n states (subset construction worst case).

Memory Trick

- **"Thompson builds NFA, Subset makes it DFA."**

CBT Trap

- **NFA and DFA accept the SAME class** of languages (**regular**); NFA is not more powerful, only more compact.
- The scanner runs as a **DFA** for speed; matching uses the **longest match (maximal munch)** rule and a **priority** rule for ties (keywords beat identifiers).

2.5 Input Buffering

- To reduce I/O overhead, scanners read input into buffers; the classic scheme uses **two buffers** of size **N** each with a **sentinel** (eof) to halve the end-of-buffer tests.
- Two pointers are used: **lexemeBegin** and **forward**.

CBT Trap

- The **sentinel** character (often eof) avoids testing for end-of-buffer at every character read.

2.6 LEX / Flex Tool

- **LEX** (and GNU **Flex**) is a **lexical-analyzer generator**: you give regex patterns and actions, it emits C code for the scanner.
- A LEX program has **3** sections separated by **%**: **declarations**, **translation rules**, and **auxiliary functions**.
- Output is a function commonly named `yyLex()`.

CBT Trap

- LEX = scanner generator; **YACC** = parser generator. Do not swap them.
- On overlapping matches LEX picks the **longest match**; on equal length it picks the **rule listed first**.

2.7 Symbol Table Interaction

- When the scanner finds an **identifier**, it enters/looks it up in the **symbol table** and returns a pointer as the token's attribute.
- Keywords are usually **pre-loaded** into the symbol table or checked against a reserved list so they are not treated as identifiers.

CBT Trap

- Returning the **symbol-table pointer** as the attribute (not the raw string) is the standard design.

3. Grammars for Parsing

3.1 Context-Free Grammars (CFG)

- A **CFG** is a 4-tuple $G = (V, T, P, S)$: **V** = non-terminals (variables), **T** = terminals, **P** = productions, **S** = start symbol.
- Each production has the form $A \rightarrow \alpha$ where **A** is a single non-terminal and **alpha** is a string of terminals/non-terminals.
- CFGs generate **context-free languages** and can express **nested/recursive** structures (balanced parentheses, block nesting) that regex cannot.

CBT Trap

- The LHS of a CFG production has **exactly one non-terminal**; if more, it is **context-sensitive**, not context-free.
- Programming-language **syntax** is described by **CFG (Type-2)**; **tokens** by regular grammar (Type-3).

3.2 Parse Trees and Derivations

- A **derivation** is a sequence of production applications starting from **S**; **leftmost derivation** expands the leftmost non-terminal first, **rightmost** the rightmost.
- A **parse tree** is a graphical view of a derivation: root = **start symbol**, leaves = **terminals/yield**, internal nodes = non-terminals.
- Top-down** parsers trace a **leftmost** derivation; **bottom-up** parsers trace a **rightmost derivation in reverse**.

CBT Trap

- One parse tree can correspond to **many** derivations, but a **leftmost** (or rightmost) derivation is **unique** per parse tree.
- Bottom-up = **rightmost derivation in reverse** (very common MCQ).

3.3 Ambiguity, Left Recursion, Left Factoring

- A grammar is **ambiguous** if some string has **two or more** distinct parse trees (equivalently, two leftmost derivations).
- Ambiguity is undecidable** in general; we remove it by rewriting (precedence/associativity rules) - e.g., the **dangling-else** problem (match else to nearest if).
- Left recursion**: $A \rightarrow A \alpha \mid \beta$ causes infinite loop in top-down parsing; eliminate to $A \rightarrow \beta A'$, $A' \rightarrow \alpha A' \mid \epsilon$.
- Left factoring**: when two productions share a common prefix, factor it: $A \rightarrow \alpha \beta_1 \mid \alpha \beta_2$ becomes $A \rightarrow \alpha A'$, $A' \rightarrow \beta_1 \mid \beta_2$ (needed for **LL(1)**).

Left Recursion Removal Formula

- General $A \rightarrow A a_1 \mid A a_2 \mid \dots \mid b_1 \mid b_2$ becomes $A \rightarrow b_1 A' \mid b_2 A'$ and $A' \rightarrow a_1 A' \mid a_2 A' \mid \epsilon$.

CBT Trap

- Left recursion** kills **top-down** parsers but is FINE for **bottom-up** (LR) parsers - LR actually prefers left recursion.
- Ambiguity is a property of the **grammar**, not the language; an **inherently ambiguous** language has no unambiguous grammar.

- Removing left recursion does NOT remove **ambiguity**; they are different problems.

4. Top-Down Parsing

4.1 Recursive Descent Parsing

- **Recursive descent** uses one **recursive procedure per non-terminal**; it builds the parse tree **top-down, leftmost**.
- General recursive descent may need **backtracking**; the grammar must be free of **left recursion**.

CBT Trap

- **Predictive parsing** is recursive descent WITHOUT backtracking (uses **lookahead** to choose the production).

4.2 Predictive Parsing LL(1)

- **LL(1)**: **L** = scan **Left-to-right**, **L** = **Leftmost** derivation, **1** = **one** symbol of lookahead.
- It is **table-driven** and **non-backtracking**; uses an explicit **stack** + **parsing table** $M[A, a]$.
- Requires the grammar to be **non-left-recursive** and **left-factored**.

4.2.1 FIRST Set Computation

- **FIRST(X)** = set of terminals that can **begin** a string derived from X (may include **epsilon**).
- Rules: if X is terminal, $\text{FIRST}(X) = \{X\}$; if $X \rightarrow \epsilon$, add **epsilon**; for $X \rightarrow Y_1 Y_2 \dots Y_k$, add $\text{FIRST}(Y_1) \setminus \{\epsilon\}$, and continue to Y_2 only if **epsilon in FIRST(Y1)**, etc.

4.2.2 FOLLOW Set Computation

- **FOLLOW(A)** = set of terminals that can appear **immediately to the right** of A in some sentential form.
- Rules: put **\$** (end marker) in $\text{FOLLOW}(\text{start symbol})$; for $A \rightarrow \alpha B \beta$, add $\text{FIRST}(\beta) \setminus \{\epsilon\}$ to $\text{FOLLOW}(B)$; if **epsilon in FIRST(beta)** or β is empty, add **FOLLOW(A)** to $\text{FOLLOW}(B)$.

4.2.3 LL(1) Parsing Table Construction

- For each production $A \rightarrow \alpha$: for every terminal a in $\text{FIRST}(\alpha)$, put $A \rightarrow \alpha$ in $M[A, a]$.
- If **epsilon in FIRST(alpha)**, then for every b in $\text{FOLLOW}(A)$, put $A \rightarrow \alpha$ in $M[A, b]$ (and include **\$** if it is in $\text{FOLLOW}(A)$).

4.2.4 LL(1) Grammar Conditions

- A grammar is **LL(1)** iff for every non-terminal with productions $A \rightarrow \alpha \mid \beta$: (1) **$\text{FIRST}(\alpha) \cap \text{FIRST}(\beta) = \emptyset$** ; (2) at most one of them derives epsilon; (3) if $\beta \Rightarrow^* \epsilon$ then **$\text{FIRST}(\alpha) \cap \text{FOLLOW}(A) = \emptyset$** .
- Equivalently, **no cell of M has two entries** (no multiple-defined entry).

FIRST/FOLLOW Quick Table

Set	Question it answers	Includes epsilon?
FIRST(X)	What can start X?	Yes possible
FOLLOW(A)	What can follow A?	Never (uses \$ instead)

Memory Trick

- "**FIRST starts, FOLLOW finishes.**" FOLLOW never contains epsilon but may contain **\$**.

CBT Trap

- **Every LL(1) grammar is unambiguous**, but not every unambiguous grammar is LL(1).
- A grammar with **left recursion** or **common prefixes** is **NOT** LL(1) until fixed.
- **Epsilon** can be in FIRST but **never** in FOLLOW; **\$** can be in FOLLOW but never in FIRST.

4.3 Backtracking Parsers

- A **backtracking** top-down parser tries a production and, on failure, **undoes** input consumption and tries the next alternative.
- It is **slow** (potentially exponential) and cannot easily handle **semantic actions/side effects**; avoided in practice.

CBT Trap

- Backtracking is needed only when **lookahead** is insufficient; LL(1) avoids it entirely.

5. Bottom-Up Parsing

5.1 Handles and Handle Pruning

- A **handle** is a substring that matches the **right side of a production** AND whose reduction represents one step of a **rightmost derivation in reverse**.
- **Handle pruning** = repeatedly finding a handle and **reducing** it to its LHS until the start symbol is reached.

CBT Trap

- A handle is not just any RHS match; it must be the **correct** one for a rightmost derivation - position matters.

5.2 Shift-Reduce Parsing and Conflicts

- **Shift-reduce** parsing uses a **stack** and **4 actions**: **shift**, **reduce**, **accept**, **error**.
- **Shift**: push next input token; **Reduce**: replace a handle on top of stack by its LHS; **Accept**: success; **Error**: syntax error.
- **Shift/reduce conflict**: parser cannot decide whether to shift or reduce (e.g., **dangling else**).
- **Reduce/reduce conflict**: parser cannot decide which of two productions to reduce by.

CBT Trap

- **Dangling-else** causes a **shift/reduce** conflict; default resolution = **shift** (matches else to nearest if).

5.3 LR Parsing Family

- **LR**: **L** = Left-to-right scan, **R** = **Rightmost** derivation in reverse; the most powerful practical class.
- Four members in increasing power: **LR(0) < SLR(1) < LALR(1) < CLR(1)/LR(1)**.

5.3.1 LR(0) Items and Automaton

- An **LR(0) item** is a production with a **dot** marking position, e.g., $A \rightarrow X \cdot Y Z$.
- **Closure** and **GOTO** operations build the **canonical collection** of LR(0) item sets = states of the **DFA** (the **LR(0) automaton**).
- LR(0) reduces on a state with a **complete item** $A \rightarrow \alpha \cdot$ regardless of lookahead.

5.3.2 SLR(1) Parsing Table

- **SLR(1)** (Simple LR) uses LR(0) items but places a **reduce** action $A \rightarrow \alpha \cdot$ only on terminals in **FOLLOW(A)**.
- It is the **weakest** of the lookahead LR methods; the smallest tables among LR(1)-style methods (same states as LR(0)).

5.3.3 CLR(1) / LR(1) Items and Table

- An **LR(1) item** adds a **lookahead** terminal: $[A \rightarrow \alpha \cdot \text{beta}, a]$.
- **Canonical LR (CLR)** is the **most powerful**; it has the **largest** number of states/table size.

5.3.4 LALR(1) Parsing Table

- **LALR(1)** merges CLR states that have the **same core** (same LR(0) items, differing only in lookahead).
- Result: **same number of states as SLR/LR(0)** but more power than SLR; merging can introduce **reduce/reduce** conflicts (never new shift/reduce).

- Used by **YACC/Bison** - the practical sweet spot.

LR States Count

- States: **LR(0) = SLR = LALR** (same count) **< CLR** (more states).

5.4 Comparison of LL, SLR, LALR, CLR Power

Parser	Direction/Deriv	Lookahead	Power	Table size
LL(1)	Left/Leftmost	1	Weakest	Small
LR(0)	Left/Rightmost-rev	0	Low	Small
SLR(1)	Left/Rightmost-rev	1 (FOLLOW)	> LR(0)	Small
LALR(1)	Left/Rightmost-rev	1 (merged)	> SLR	Small
CLR(1)	Left/Rightmost-rev	1 (full)	Strongest	Largest

- Power ordering: **LR(0) < SLR(1) < LALR(1) < CLR(1)**; and **LL(1) is a subset of LR(1)** in language power.
- Every LL(1) grammar is LR(1), but **not** vice versa.

Memory Trick

- Power ladder: "**LR0 < SLR < LALR < CLR**" - alphabetical-ish climb; CLR is king but heaviest.
- States: "**SLR = LALR = LR0**" in count; only **CLR** blows up.

CBT Trap

- **LALR** can have **reduce/reduce** conflicts that **CLR** does not (because of state merging); LALR never adds **shift/reduce** conflicts beyond CLR.
- If a grammar is **LR(0)**, it is automatically **SLR/LALR/CLR**.
- Number of **states**: CLR $\geq$ the others; do not say CLR has fewer states.

5.5 Operator Precedence Parsing

- **Operator-precedence** parsing is a shift-reduce method for **operator grammars** (no epsilon, no two adjacent non-terminals).
- Uses **precedence relations** **<, ., >, =** between terminals to find handles.

CBT Trap

- Operator grammars forbid **two adjacent non-terminals** on any RHS and forbid **epsilon** productions.

5.6 YACC / Bison Tool

- **YACC** ("Yet Another Compiler-Compiler") / GNU **Bison** generates an **LALR(1)** parser from a grammar specification.
- A YACC file has **3** sections separated by %: **declarations**, **grammar rules + actions**, **C code**.
- Default conflict resolution: **shift over reduce** for shift/reduce, and **earlier rule** for reduce/reduce.

CBT Trap

- YACC produces **LALR(1)**, not CLR; remembering this is a frequent MCQ.

6. Syntax-Directed Translation (SDT)

6.1 Syntax-Directed Definitions (SDD)

- An **SDD** attaches **attributes** and **semantic rules** to grammar productions; it specifies WHAT to compute, not the order.
- Each grammar symbol has a set of **attributes**; each production has **semantic rules**.

6.2 Synthesized vs Inherited Attributes

- A **synthesized attribute** is computed from the attributes of the node's **children** (information flows **bottom-up**).
- An **inherited attribute** is computed from the **parent and/or siblings** (information flows **top-down/sideways**).
- **Terminals** can have only **synthesized** attributes (supplied by the lexer).

Synthesized vs Inherited Table

Feature	Synthesized	Inherited
Computed from	Children	Parent + siblings
Flow	Bottom-up	Top-down/sideways
Bottom-up parsers	Easy	hard
Example	type of an expr	type passed to a var list

CBT Trap

- Terminals carry only **synthesized** attributes; the **start symbol** cannot have an **inherited** attribute (no parent).

6.3 Annotated Parse Trees

- An **annotated (decorated) parse tree** shows the **attribute values** at each node after applying the semantic rules.

6.4 S-Attributed and L-Attributed Definitions

- **S-attributed**: uses **only synthesized** attributes; can be evaluated **bottom-up** during **LR** parsing; actions at **end** of RHS.
- **L-attributed**: each inherited attribute of a symbol depends only on **attributes to its LEFT** (and inherited attributes of the parent); can be evaluated **left-to-right** in one depth-first pass, fits **LL** parsing.
- Every **S-attributed** SDD is also **L-attributed** (S is a subset of L).

CBT Trap

- All **S-attributed** are **L-attributed**, but NOT vice versa.
- **S-attributed** <-> bottom-up/LR; **L-attributed** <-> top-down/LL with one left-to-right pass.

6.5 Translation Schemes

- A **translation scheme** is an SDD with **semantic actions** embedded (in { }) at specific positions in the RHS.
- For **S-attributed**, actions go at the **end**; for **L-attributed**, an inherited attribute action must appear **before** the symbol that uses it.

CBT Trap

- Placing actions in the wrong position breaks **L-attributed** evaluation; an action using an inherited attribute must precede that symbol.

6.6 Dependency Graphs

- A **dependency graph** has an edge from attribute a to b if b depends on a; a valid evaluation order is a **topological sort**.
- If the dependency graph has a **cycle**, no evaluation order exists (SDD is not well-defined).

CBT Trap

- A **cyclic** dependency graph means the attributes **cannot** be evaluated; evaluation order = **topological order**.

7. Semantic Analysis

7.1 Type Checking and Type Systems

- **Type checking** verifies that operations receive operands of **compatible types**; it uses a **type system** (rules + type expressions).
- **Static type checking** happens at **compile time**; **dynamic** at **run time**.
- A **strongly typed** language guarantees no type errors slip to runtime; a **type expression** can be basic (int, char) or constructed (array, record, pointer, function).

CBT Trap

- **Type checking** is the headline job of **semantic analysis**, done at **compile time** for static languages.

7.2 Type Conversions / Coercion

- **Coercion** = **implicit** type conversion done by the compiler (e.g., **int** -> **float** in `2 + 3.5`).
- **Casting** = **explicit** conversion requested by the programmer.
- **Widening** (smaller -> larger, safe) vs **narrowing** (larger -> smaller, possible data loss).

CBT Trap

- **Coercion** = **implicit**; **cast** = **explicit**. Widening is automatic; narrowing usually needs an explicit cast.

7.3 Scope Checking

- **Scope** is the region of program where a name binding is valid; **static (lexical) scope** resolves names by **program text/nesting**, **dynamic scope** by **call sequence**.
- The compiler verifies **declaration before use** and resolves names to the **innermost** enclosing declaration.

CBT Trap

- Most languages (C, Java) use **static/lexical** scope; **dynamic** scope (older Lisp) resolves by call stack.

7.4 Symbol Table Organization and Operations

- Operations: **insert**, **lookup**, **delete** (on scope exit); for nested scopes use a **stack of tables** or scope-numbered entries.
- Implementations and lookup cost: **linear list** $O(n)$, **BST** $O(\log n)$ average, **hash table** **$O(1)$** average.

CBT Trap

- **Hash table** gives average **$O(1)$** ; worst case can be $O(n)$ with collisions.

8. Runtime Environments

8.1 Storage Organization

- Runtime memory is divided into: **Code (text)** for instructions, **Static** for globals/static data, **Heap** for dynamic allocation, and **Stack** for activation records.
- Typical layout: **code** at low addresses, then **static**, then **heap growing up**, and **stack growing down** from high addresses (they grow toward each other).

CBT Trap

- Stack** grows **downward**, **heap** grows **upward** (toward each other) in the common model.

8.2 Activation Records (Stack Frames)

- An **activation record (frame)** holds info for one procedure call; contents: **return value**, **actual parameters**, **control link (dynamic)**, **access link (static)**, **saved machine status**, and **local data + temporaries**.
- A new frame is **pushed** on call and **popped** on return.

Activation Record Contents Table

Field	Purpose
Return value	result to caller
Actual parameters	arguments
Control link	points to caller's frame (dynamic)
Access link	for non-local data (static)
Saved status	return address, registers
Locals/temps	local variables

CBT Trap

- Control/dynamic link** -> caller's frame; **access/static link** -> lexically enclosing scope's frame. Do not swap them.

8.3 Stack Allocation; Heap Management

- Stack allocation**: frames pushed/popped with calls/returns; ideal for languages without persistence of locals after return.
- Heap**: for data that outlives the procedure (dynamic allocation); needs **garbage collection** or manual free/delete.

CBT Trap

- Recursion** requires **stack** allocation (each call gets its own frame); pure **static** allocation cannot support recursion.

8.4 Access to Non-Local Names

- Static scope**: a name refers to the declaration in the nearest enclosing block in the **program text**; uses **access (static) links** or a **display**.
- Dynamic scope**: a name refers to the most recent binding in the **call chain**; uses the **control link** chain.

CBT Trap

- **Access link** = static scope navigation; **control link** = dynamic/return navigation.

8.5 Parameter Passing

- **Call by value**: copy of the argument; callee changes do NOT affect caller.
- **Call by reference**: address passed; callee changes DO affect caller.
- **Call by value-result (copy-restore)**: copy in on entry, copy back on return.
- **Call by name**: argument re-evaluated each use (textual substitution / thunks); rare.

Parameter Passing Table

Method	Affects caller?	Note
Value	No	copy in
Reference	Yes	address
Value-result	Yes (on return)	copy-restore
Name	Yes	re-evaluated each use

CBT Trap

- **Call by value** cannot swap two variables; **call by reference** can.
- **Call by name** can give different results from value-result when arguments have **side effects** or array subscripts change.

9. Intermediate Code Generation

9.1 Intermediate Representations

- Common IRs: **syntax trees**, **DAGs** (Directed Acyclic Graphs), and **postfix (Polish) notation**.
- A **DAG** is like a syntax tree but **shares** common subexpressions (no duplicate nodes) - basis for **CSE**.
- Postfix** notation needs no parentheses and is evaluated with a **stack**.

CBT Trap

- A **DAG** differs from a syntax tree by **sharing** common subexpressions; it exposes **common subexpressions** for optimization.

9.2 Three-Address Code (TAC)

- TAC** statements have at most **three addresses** and **one operator**: general form $x = y \text{ op } z$.
- Each instruction has at most **one** operator on the RHS; complex expressions are broken into temporaries $t_1, t_2, \dots$
- Common forms: assignment, copy $x = y$, unconditional goto L , conditional $\text{if } x \text{ relop } y \text{ goto } L$, param, call.

CBT Trap

- "**Three-address**" counts up to **two operands + one result** = three addresses; a single statement has at most **one** operator.

9.3 Representations of TAC: Quadruples, Triples, Indirect Triples

- Quadruple**: 4 fields (**op, arg1, arg2, result**); uses explicit temporary names; easy to **move/reorder** and optimize.
- Triple**: 3 fields (**op, arg1, arg2**); the result is referred to by the **triple's position/index** (no explicit temp).
- Indirect triple**: a list of **pointers** to triples; reordering optimizations move pointers, not triples.

Quad vs Triple vs Indirect Table

Form	Fields	Temp names?	Reordering
Quadruple	4 (op,arg1,arg2,result)	Yes	Easy
Triple	3 (op,arg1,arg2)	No (by position)	Hard
Indirect triple	pointers + triples	No	Easy

Memory Trick

- "**Quad = 4 fields with result**; Triple = 3, result is the index; Indirect = pointers to triples."

CBT Trap

- Triples** are hard to **reorder** because positions are referenced directly; **indirect triples** fix this with a pointer list.
- Quadruples** make **temporary** reuse and code motion easiest.

9.4 Translation of Expressions and Assignments

- For $a = b + c * d$, generate $t_1 = c * d$, $t_2 = b + t_1$, $a = t_2$ (respecting **precedence**).

- SDT attributes: each non-terminal E has E.place (the name holding the value) and E.code (the generated code).

9.5 Translation of Control Flow and Boolean Expressions

- **if**, **while**, **for** translate into **conditional** and **unconditional** jumps with labels.
- Boolean expressions use **short-circuit (jumping) code**: **&&** and **||** translate to jumps, evaluating only as far as needed.

CBT Trap

- **Short-circuit** evaluation means A **&&** B skips B when A is false; A **||** B skips B when A is true.

9.6 Backpatching

- **Backpatching** fills in jump targets that are unknown at first generation by keeping **lists** (truelist, falselist, nextlist) and patching them once the target label is known - enables **one-pass** code generation.
- Functions: **makelist**, **merge**, **backpatch**.

CBT Trap

- Backpatching lets boolean/flow code be generated in **one pass** despite forward jumps.

9.7 Translation of Arrays and Function Calls

- Array element address = **base + index * width** (1-D); for multi-dimension use row-major/column-major formulas.
- Row-major address for A[i][j] (bounds 0..) = **base + (i * n2 + j) * w** where n2 = number of columns.
- Function calls translate to **param** statements followed by a **call** statement.

CBT Trap

- **Row-major** (C) stores rows contiguously; **column-major** (Fortran) stores columns contiguously - address formulas differ.

10. Local & Machine-Independent Optimization

10.1 Basic Blocks and Flow Graphs

- A **basic block** is a maximal sequence of consecutive statements with **one entry** (first statement) and **one exit** (last statement); no branching in/out except at ends.
- **Leaders** (first statements of blocks): the **first** statement, any **target of a jump**, and any statement **immediately following a jump**.
- A **flow graph** has basic blocks as nodes and **edges** for possible control flow between them.

Leader Rules

- A statement is a **leader** if it is: (1) the **first** statement; (2) the **target** of a conditional/unconditional jump; (3) the statement **right after** a jump.

CBT Trap

- To count basic blocks, first find **leaders**; the number of leaders = number of basic blocks.

10.2 Local Optimizations

10.2.1 Constant Folding

- **Constant folding**: evaluate constant expressions at **compile time**, e.g., $x = 3 * 4 \rightarrow x = 12$.

10.2.2 Constant Propagation

- **Constant propagation**: replace a variable known to hold a constant with that constant, e.g., if $x = 5$ then $y = x + 2 \rightarrow y = 5 + 2$.

10.2.3 Copy Propagation

- **Copy propagation**: after a copy $x = y$, replace later uses of x with y (enables dead-code removal of the copy).

10.2.4 Dead Code Elimination

- **Dead code elimination**: remove statements whose results are **never used** (dead variables) or unreachable code.

10.2.5 Common Subexpression Elimination (CSE)

- **CSE**: if an expression $b + c$ is already computed and operands unchanged, reuse the prior result instead of recomputing.
- Local CSE uses a **DAG**; global CSE uses **available expressions** data flow.

10.2.6 Algebraic Simplification / Strength Reduction

- **Algebraic simplification**: apply identities, e.g., $x + 0 = x$, $x * 1 = x$, $x * 0 = 0$.
- **Strength reduction**: replace expensive operations with cheaper ones, e.g., $x * 2 \rightarrow x \ll 1$, multiplication in loops $\rightarrow$ addition.

Optimization Examples Table

Optimization	Before	After
Constant folding	<code>a = 2 * 3</code>	<code>a = 6</code>
Constant propagation	<code>x=5; y=x+1</code>	<code>y=6</code>
Copy propagation	<code>x=y; z=x</code>	<code>z=y</code>
Strength reduction	<code>a = b * 2</code>	<code>a = b << 1</code>
Dead code	<code>x=5; (x unused)</code>	removed
CSE	<code>a=b+c; d=b+c</code>	<code>d=a</code>

CBT Trap

- **Constant folding** (fold a constant **expression**) vs **constant propagation** (substitute a **variable's** constant value) - examiners swap these.
- **Strength reduction** swaps **operator cost** (mul->add/shift); it does NOT delete code.

10.3 Loop Optimizations

- **Code motion (loop-invariant code motion)**: move computations that do not change inside the loop **out of the loop**.
- **Induction variables**: variables that change by a **constant** each iteration; can be **reduced in strength** or **eliminated**.
- **Loop unrolling** reduces loop overhead; **loop fusion** merges adjacent loops.

CBT Trap

- Loop-invariant code motion is **machine-independent**; moving `y = x * x` out of a loop where x is invariant is the canonical example.

10.4 Peephole Optimization

- **Peephole optimization** examines a **small sliding window** (peephole) of target instructions and replaces inefficient patterns.
- Techniques: **redundant load/store elimination**, **algebraic simplification**, **unreachable code** removal, **flow-of-control** optimization, **strength reduction**.

CBT Trap

- Peephole is typically a **machine-dependent**, **local** optimization done on **target code** over a **small window**.

11. Data Flow Analysis

11.1 Reaching Definitions

- A **definition** d **reaches** a point p if there is a path from d to p with no **redefinition** of that variable in between.
- It is a **forward, may (union)** analysis; used for **constant propagation** and detecting uninitialized use.
- Equation: $OUT[B] = gen[B] \cup (IN[B] - kill[B])$; $IN[B] = \cup OUT[predecessors]$.

11.2 Live Variable (Liveness) Analysis

- A variable is **live** at a point if its current value is **used later** along some path before redefinition.
- It is a **backward, may (union)** analysis; used for **register allocation** and **dead-code elimination**.
- Equation: $IN[B] = use[B] \cup (OUT[B] - def[B])$; $OUT[B] = \cup IN[successors]$.

11.3 Available Expressions

- An expression $x \ op \ y$ is **available** at p if EVERY path to p computes it and no operand is redefined afterward.
- It is a **forward, must (intersection)** analysis; used for **global CSE**.
- Equation: $IN[B] = intersect \ OUT[predecessors]$; $OUT[B] = e_gen[B] \cup (IN[B] - e_kill[B])$.

11.4 Constant Propagation Analysis

- A data-flow framework that determines whether a variable has a **constant** value at each point; uses a **lattice** with values **UNDEF, constant, NAC (not-a-constant)**.
- It is a **forward** analysis; the lattice meet handles conflicting constants $\rightarrow$ **NAC**.

11.5 Use of CSE via Data Flow

- Global **CSE** uses **available expressions**: if $b + c$ is available at a point, the recomputation is replaced by the stored value.

Data Flow Analysis Master Table

Analysis	Direction	Meet (confluence)	Use
Reaching definitions	Forward	Union (may)	const prop
Available expressions	Forward	Intersection (must)	global CSE
Live variables	Backward	Union (may)	reg alloc, dead code
Constant propagation	Forward	lattice meet	folding

Memory Trick

- "**Reaching** and **Available** go **forward**; **Live** goes **backward**." "**Available** is the only **intersection (must)**; the may-analyses use **union**."

CBT Trap

- Live variables** = **backward + union**; **Available expressions** = **forward + intersection**. These two are the most-confused pair.
- Reaching definitions** uses **union** (may); **available expressions** uses **intersection** (must) - both forward, different meet.

12. Code Generation

12.1 Issues in Code Generation

- Key issues: **instruction selection**, **register allocation and assignment**, **instruction ordering/scheduling**, and the **target machine model**.
- Good code generation must preserve **semantics** while exploiting machine features (addressing modes, instruction costs).

CBT Trap

- **Register allocation** (which values live in registers) vs **register assignment** (which specific register) - distinct sub-steps.

12.2 Register Allocation and Spilling

- **Register allocation** chooses which variables reside in **limited registers**; classic method models it as **graph coloring** of the **interference graph**.
- **Spilling** = when registers run out, store a value to **memory** and reload later (costly).

CBT Trap

- **Graph coloring** register allocation is **NP-complete** in general; a graph that is **k-colorable** fits in k registers.

12.3 Simple Code Generation from TAC

- A **simple code generator** processes one basic block, tracking **register descriptors** (what each register holds) and **address descriptors** (where each variable's value is).
- The **getReg** function decides the register for an operation.

12.4 DAG-Based Local Code Generation

- Building a **DAG** for a basic block exposes **common subexpressions** and lets the generator pick a good **evaluation order** and reduce recomputation.
- **Labeling (Sethi-Ullman)** algorithm computes the **minimum number of registers** needed to evaluate an expression tree without spilling.

CBT Trap

- **Sethi-Ullman** labeling gives the **minimum registers** for an expression tree (no stores); the label of a leaf is **1** (or 0 for a right leaf), interior label combines children.

13. Exam Focus / Practice Checklist

13.1 Phase Identification

- Match each error to its phase: **invalid token** -> **lexical**, **missing ;** -> **syntax**, **type mismatch** -> **semantic**, **undeclared variable** -> **semantic**.
- Know which phase produces which output: **tokens**, **parse tree**, **annotated tree**, **IR**, **optimized IR**, **machine code**.

13.2 FIRST and FOLLOW Computation

- Practice computing **FIRST** and **FOLLOW** for small grammars; remember **epsilon** only in FIRST and **\$** only in FOLLOW (of start symbol at minimum).

13.3 LL(1) / LR Conflict Detection

- An **LL(1)** table conflict = a **cell with 2 entries**; an **LR** conflict = **shift/reduce** or **reduce/reduce** in a state.

13.4 Parser-Type Power Comparison

- Memorize **LR(0) < SLR(1) < LALR(1) < CLR(1)** for power, equal states for **SLR=LALR=LR0**, and **YACC = LALR(1)**.

13.5 Synthesized vs Inherited

- Identify whether an attribute flows from **children (synthesized)** or **parent/siblings (inherited)**; S-attributed $\subseteq$ L-attributed.

13.6 Quadruples/Triples

- Be able to write **quadruples** and **triples** for an expression like $a = b * c + d$.

13.7 Number of Tokens

- Count tokens precisely: keywords, identifiers, constants, operators, and separators each count as tokens; whitespace/comments do not.

13.8 Definitions and One-Liners

- Keep crisp definitions of **token**, **handle**, **basic block**, **leader**, **DAG**, **activation record**, **backpatching** ready for direct recall.

Final Memory Trick

- Phases: **"La Sa Se In Op Co"**; Parser power: **"LR0 < SLR < LALR < CLR"**; Data flow: **"Live=backward/union, Available=forward/intersection"**.